DEPARTMENT 1 · DEVELOPMENT AND IT · PROFESSION 5

AI developer

Profession 5 of 10 in Development and IT.

What the profession means

An AI developer builds applications that use AI models at their core: assistants, document tools, generators and agents. The developer writes the application code, the model calls, the data handling and the user interface.

What professionals do every day

Designs the feature, writes prompts and evaluation sets, builds the application and API layer, adds retrieval where documents matter, tests quality and deploys with monitoring and cost limits.

Why businesses hire them, and what problems they solve

Businesses want AI features inside their products without building models from scratch. An AI developer delivers these features reliably, safely and at a predictable cost.

- Feature ideas that need a working prototype.
- Document-heavy work that is slow to read and answer.
- Customer-facing AI that must be accurate and safe.

Skills you need

- Python or JavaScript application development.
- Model APIs, structured outputs and function calling.
- RAG and vector search (see Module D).
- Evaluation: test sets, scoring and regression checks.
- Security, privacy and cost monitoring.

Tools (free and paid)

Tool	What it is for	Free option	Paid option	Official link (not checked live)
OpenAI API	Models and tools	Pay per use	Pay per use	platform.openai.com/docs
Anthropic API	Claude models and tools	Pay per use	Pay per use	docs.claude.com
Google AI Studio	Gemini models	Free tier	Pay per use	aistudio.google.com
Cursor	Code editor with AI	Free tier	Paid plans	cursor.com
Vercel	Hosting for AI apps	Hobby tier	Paid plans	vercel.com

Learning sequence: beginner, intermediate, advanced

Level	What you can do when you finish
Beginner	A single-page app that calls a model and shows the result.
Intermediate	A document question-answer app with sources.

Advanced	An app with login, usage limits and an evaluation report.

Projects by level

- **Beginner:** Summary tool for meeting notes.
- **Intermediate:** Q&A over a company handbook.
- **Advanced:** Proposal writer with a review step and history.

Client-ready portfolio project

A document assistant for a fictional firm: upload or paste documents, ask questions, get sourced answers, and see an evaluation report on a test set.

Worked example

Worked example. A question goes to a retriever that returns the best passages; the model is asked to answer only from them and cite the page. The app shows the passages next to the answer so the user can check.

Assignment with full solution

ASSIGNMENT

Evaluate an AI feature before release

Business problem: A team wants to ship a document Q&A feature. They need evidence it answers correctly, and they must know when it fails.

Steps

1. Write 20 test questions with the correct answer and the source page.
2. Run the feature on each question and record whether the answer is correct and cited.
3. Count the failures and group them by cause (retrieval miss, wrong fact, no answer).
4. Fix one cause and rerun the full set to check nothing else broke.

Expected output: A table of results with a pass rate, failure causes and a before-and-after comparison.

SOLUTION (ANSWER KEY)

Evaluation makes quality visible. Retrieval misses and wrong facts need different fixes. Rerunning the whole set after each change catches regressions. The RAG example in Module D is a tiny version of this loop.

PYTHON

```
import re
from collections import Counter

STOP = {"the", "and", "for", "are", "you", "your", "how", "what", "does", "long", "take",
        "have", "with", "from", "that", "this", "after", "within", "each", "shown", "before",
"run", "our", "can", "will"}
DOCS = {
    "refund": "Refunds are processed within 7 working days after a written request.",
```

```
    "fees": "Course fees are shown on each program page and confirmed in writing before
payment.",
    "timings": "Live classes run on Saturdays and Sundays in the evening.",
}

def tokens(text):
    return [w for w in re.findall(r"[a-z]+", text.lower()) if len(w) > 2 and w not in STOP]

def retrieve(question, docs=DOCS, k=1):
    q = Counter(tokens(question))
    scored = []
    for key, text in docs.items():
        d = Counter(tokens(text))
        score = sum(min(q[w], d[w]) for w in q)
        scored.append((score, key, text))
    scored.sort(reverse=True)
    return [(key, text) for score, key, text in scored[:k] if score > 0]

def answer(question):
    hits = retrieve(question)
    if not hits:
        return "Not found in our documents. A team member will reply."
    key, text = hits[0]
    return f"{text} [source: {key}]"

if __name__ == "__main__":
    assert answer("How long do refunds take?").endswith("[source: refund]")
    assert "7 working days" in answer("How long do refunds take?")
    assert answer("What are the fees for the course?").endswith("[source: fees]")
    assert answer("When do the live classes run?").endswith("[source: timings]")
    assert answer("Do you sell pizza?").startswith("Not found")
    print("rag tests passed")
```

Common mistakes

- Shipping without an evaluation set.
- Trusting one good demo.
- Sending whole private documents to a model without checking the provider's data terms.
- Ignoring cost per request.

Freelancing opportunities

- AI features for small software products.
- Document assistants for law, property and education firms (with confidentiality checks).
- Freelance briefs for 'AI app', 'OpenAI', 'Claude API', 'RAG'.

Learning resources

LEARNING RESOURCES (VERIFY BEFORE USE)

- OpenAI API documentation
- Claude documentation
- Google AI for developers

Part of **Master Freelancing: AI and Automation**. Back to the chapter list on skillnavora.in · Open the full textbook. Official links in this chapter were not checked live; confirm before relying on them.